

Decorators — Basics

Wrapping functions with extra power, the Flask/Django way

■ What is a Decorator?

Yesterday you mastered how Generators use `yield` to stream data one item at a time. Today, we're diving into **Decorators** — a technique used everywhere in Python frameworks like Flask, Django, and FastAPI!

A Decorator is a function that takes another function as an argument, adds some extra functionality or behavior to it, and returns it — without modifying the original function's code.

■ Think of a Decorator Like a Gift Wrapper

The present inside is your original function. The gift wrap is the decorator — it adds decoration on the outside without changing what's inside the box!

■ Functions Are "First-Class Objects"

Before writing a decorator, you need to know two key rules about Python functions:

1. Functions can be assigned to variables

```
def greet():  
    return "Hello!"  
  
say_hi = greet  # Assigning function to a variable  
print(say_hi()) # Output: Hello!
```

2. Functions can be passed into other functions as arguments

```
def run_function(func):  
    print("Running now...")  
    func() # Calls whatever function was passed in!  
  
run_function(greet)
```

■■ The @ Syntax (The Magic Wrapper)

Instead of manually wrapping functions, Python gives us the clean `@decorator_name` syntax.

Day 76 — Decorators: Basics

How a Basic Decorator Works

```
# 1. The Decorator Function
def my_decorator(func):
    def wrapper():
        print("--- [BEFORE] Code running before original function ---")
        func() # Calls the original function
        print("--- [AFTER] Code running after original function ---")
    return wrapper

# 2. Applying the Decorator using @
@my_decorator
def say_hello():
    print("■ Hello Shoaib!")

# 3. Call the decorated function
say_hello()
```

■ Output Preview

```
--- [BEFORE] Code running before original function ---
■ Hello Shoaib!
--- [AFTER] Code running after original function ---
```

■■ Real-World Use Case: Measuring Execution Time

In backend development, decorators are commonly used for logging, authentication checks, and measuring performance. Let's build one that wraps a function with extra behavior — create **backend/day76.py** in VS Code:

Step-by-Step Code Example

Day 76 — Decorators: Basics

```
# =====
# 1. THE DECORATOR (The Outfit / Wrapper)
# =====
def party_outfit(original_function):

    # This inner function 'wrapper' is the actual box!
    def wrapper():
        print("■ [BEFORE] Putting on a fancy party hat...")

        # Now run the actual function you handed in!
        original_function()

        print("■ [AFTER] Blowing the party horn! Party is over!")

    return wrapper # Hand back the wrapped package!

# =====
# 2. YOUR ORIGINAL FUNCTION
# =====
@party_outfit # <--- This puts the party outfit on make_pizza!
def make_pizza():
    print("■ Making a delicious pepperoni pizza!")

# =====
# 3. CALLING THE FUNCTION
# =====
make_pizza()
```

The Output — What Happens When You Run It?

When you run `make_pizza()`, Python sees `@party_outfit` and outputs:

```
■ [BEFORE] Putting on a fancy party hat...
■ Making a delicious pepperoni pizza!
■ [AFTER] Blowing the party horn! Party is over!
```

■ Summary Checklist

Day 76 — Decorators: Basics

Concept	Kid-Friendly Analogy
Function	A robot that does one specific job.
Decorator	An outfit or wrapper you put around the robot.
@ Symbol	The glue that sticks the outfit onto the function.
Why use it?	To add extra features (like logging, timing, or security) without breaking or rewriting the original function!

■ Give It a Try!

- Copy the code above into your backend/day76.py file in VS Code.
- Run `python day76.py`.
- Once you see the party hat and pizza in your terminal, let me know if it clicked! ■

■ Remember

A decorator always follows the same shape: an outer function that takes in the original function, an inner wrapper () that calls it (adding code before and/or after), and a return wrapper at the end so Python knows what to run in its place.